

Vulnerability Detection in Solidity Smart Contracts Report

1. Introduction

This study focuses on detecting vulnerabilities in Solidity-based smart contracts using many methods like graph-based approaches, nlp approaches, ... The increasing reliance on blockchain technology has led to a rise in decentralized applications (dApps) and smart contracts. However, vulnerabilities within these contracts can result in significant financial losses and security breaches. Our methodology involves constructing Function Call Graphs (FCGs) and leveraging machine learning models to classify common vulnerabilities.

2. Data Collection Process

2.1. Dataset Sources

We utilized two primary datasets for this study:

1. **SmartBugs Wild Dataset:** A repository containing 47,398 smart contracts extracted from the Ethereum network. (Source: GitHub) [1]
2. **Slither Audited Smart Contracts:** A curated dataset available on Hugging Face, containing smart contracts analyzed by the Slither static analysis tool. [2]

2.2. Data Extraction and Preprocessing

From the **SmartBugs Wild Dataset**, we extracted functions and source code snippets identified by three tools (**Slither**, **SmartCheck**, **Mythril**) as containing vulnerabilities. The vulnerabilities of interest include:

- **Reentrancy**
- **Timestamp Dependency**
- **Unhandled Exceptions**

The vulnerability **Tx. origin** was excluded from the analysis due to its simplicity and the ease with which it can be detected by checking for the presence of the tx. origin keyword.

To ensure diversity in our dataset, we included benign functions (non-vulnerable functions) to balance the data for classification. **The extraction process was time-consuming due to the need to remove comments and apply regular expressions to identify vulnerable functions, also we still have error extracting source codes and functions when everytime we rerun the code extractor return a different number of results.**

Category	Benign	Re-Entrancy	Timestamp Dependency	Unhandled Exceptions
Source Codes	36,234	8,392	2,450	2,129
Functions	227,837	9,717	2,598	1,440

In the first line is the number of source codes and the second line is the number of functions from left to right we will have **Benign, Re-Entrancy, Timestamp Dependency, Unhandled Exceptions**. We classified a source code and function as **Benign** if they do not have any of these vulnerabilities (**Re-Entrancy, Timestamp Dependency, Unhandled Exceptions**).

2.3. Handling Duplicates

During data extraction, a **significant number of duplicate functions were identified across different vulnerabilities**. This observation **supports the hypothesis that malicious code is often copied and reused across multiple contracts with minimal modifications**.

But only use **SmartBugs Wild Dataset** show a low count of specific vulnerabilities, such as Unhandled Exceptions with only 2129 source codes, so we supplemented the data by extracting more contracts from the secondary source, that is **Slither Audited Smart Contracts**. However, we could only extract vulnerability source codes when the analysis tool in the dataset didn't mention where specifically the vulnerability was occurred.

Unchecked_Group: 1984

2.4. Final Dataset Creation

After preprocessing, we created a comprehensive dataset for each vulnerability containing:

- All source code files of a vulnerability type
- Balanced datasets with an equal number of source codes that are different vulnerabilities and benign (eg. With ReEntrancy Source Codes we randomly sample the same number of Timestamp Dependency, Unhandled Exceptions, Benign source codes and classied as non-reentrancy)

This was applied for functions as well. Each vulnerability type was stored in separate folders for ease of classification and graph generation.

3. Dataset Preparation

3.1. Create Dataset for fine-tune CodeBERT

We push all the preprocessed function datasets to a csv file

```
label
benign                226627
re_entrancy           9725
timestamp_dependency  2602
unhandled_exceptions  1448
```

After that we follow the **Final Dataset Creation** rule to create a new dataset of vulnerability for functions.

```
label
1    9725
0    9725
Name: count, dtype: int64
label
1    2602
0    2602
Name: count, dtype: int64
label
1    1448
0    1448
Name: count, dtype: int64
```

3.2. Feature Extraction with CodeBERT

We used CodeBERT, a transformer-based model pretrained on various programming languages like Java, C++, R, ... And Solidity or Smart Contract also have a similar code structure like Java so CodeBERT is likely the best option to extract embeddings from the function's code. Each function was represented as a 768-dimensional vector generated by CodeBERT. We added the extracted embeddings as node features in the FCGs.

We finetune 3 model each model used to classify a specific vulnerability type.

Results are in the table below:

	eval_loss	eval_accuracy	eval_precision	eval_recall	eval_f1
Re-entrancy	0.3825	<u>0.8422</u>	0.9042	0.7748	<u>0.8346</u>
Timestamp Dependency	0.4193	<u>0.8273</u>	0.8179	0.8545	<u>0.8358</u>
Unhandled Exceptions	0.1579	<u>0.9621</u>	0.9856	0.9384	<u>0.9614</u>

We also applying the following strategies in order to enhance richness of the feature extraction process:

- We noticed that a large number of functions exceed the max input sequence length of CodeBERT (which is 512 tokens)

Dataset	Number of Samples (>512 tokens)
Re-entrancy	1,398
Timestamp Dependency	358
Unhandled Exceptions	178

- We explored alternative transformer-based models such as **XLNet** and **Longformer** due to their ability to handle longer token sequences. **However, these models did not perform as well as CodeBERT for this task because they lacked pretraining on code-specific**

datasets, so their performance are not likely to surpass codeBERT model. Additionally, using larger models often led to GPU memory overflow on platforms like Kaggle and Google Colab.

CodeBERT is limited to 512 tokens, so to address this problem we retain the **first 128 tokens** and **last 382 tokens** of each function, following recommendations from prior research on fine-tuning BERT models for text classification [3]. **However, this approach doesn't seem to create any significant changed or noticable increase in accuracy and recall so we decide not to continue this approach.**

- **We can also change to GraphCodeBERT that have stronger capability of extracting code feature than CodeBERT.**

3.3. Graph Dataset Construction

We constructed Function Call Graphs (FCGs) for each smart contract. Each node in the graph represents a function, where edges represent function calls. To enhance the model's ability to classify vulnerabilities, we extracted the following graph-based features:

- **In-degree** and **Out-degree** of each node (2 features)
- **Katz centrality** (1 feature)
- **Closeness centrality** (1 feature)
- **Clustering coefficient** (1 feature)
- **Especially, the feature vector representation of the function code extracted by Finetuned CodeBERT model** (768 feature)

In addition to FCGs, we are also planning on applying edges feature for the source code graph for richer data representation.

4. Model Training and Comparing with SOTA methods

4.1. Trainin GNN Variants on Created Dataset

After evaluating on many variants of GNNs, we've explored that Graph Attention Network (GAT) was the model that best to apply for our graph classification task, the model was build with the following architecture:

- Two GAT layers with 1024 and 512 hidden units, respectively
- Fully connected layers: 256 → 64 → 2 (output)

This model was trained separately for each type of vulnerability.

Vulnerability Type	Epochs	Loss	Accuracy	F1 Score	Precision	Recall
Re-entrancy	19	0.0775	0.9565	0.9561	0.9440	0.9686
Timestamp Dependency	44	0.0270	0.9586	0.9561	0.9561	0.9561
Unhandled Exceptions	27	0.0051	0.9807	0.9803	0.9803	0.9803

4.2. Comparing with SOTA methods

4.2.1. Peculiar - IEEE ISSRE 2021 (A*) – Status: Stopped due to unable to access Peculiar’s Pretrained Model

- Model Overview

- The **Peculiar** approach leverages a **Transformer-based encoder**, specifically **GraphCodeBERT**, a pretrained model for programming languages. Unlike traditional models that rely on **text**, GraphCodeBERT incorporates **Data Flow Graphs (DFGs)** to retain **code structural information**, making it a more effective representation for **smart contract analysis**.
- Similar to **RoBERTa** and **BERT**, the model **processes two separate input token sequences** using a **[SEP]** token as a delimiter. However, instead of two conventional text sequences, Peculiar's input consists of:

Model Architecture. We can see the architecture of our model in Fig. 2. We follow **GraphCodeBERT [26]**, which is based on **Transformer neural architecture [38]** for programming language, and use a **Linear layer** to output the result.

Unlike **GraphCodeBERT** where the input contains data flow graph (DFG), we convert the DFG to crucial data flow graph (CDFG) and take it as the input for our model. Specifically, from the source code $SC = \{sc_1, sc_2, \dots, sc_n\}$ we first obtain the corresponding CDFG (as discussed in Section III-A) $CG(SC) = (Var, Edge)$, where $Var = \{v_1, v_2, \dots, v_k\}$ is a variable set, $Edge = \{\epsilon_1, \epsilon_2, \dots, \epsilon_l\}$ is a directed edge set indicating where the value of each variable comes from. We then concatenate the source code and variable set into a

- The raw source code
- A condensed Data Flow Graph (CDFG)
 - CDFG is a **compressed, focused version** of DFG, specifically targeting vulnerabilities such as **Re-entrancy**.
 - Reduces **context length**, optimizing computational efficiency.

- **Potential challenge: Token length limitations** due to the additional input sequence.

- Analysis of Peculiar's Approach:

- A key observation from the **Peculiar paper** is its similarity to **TMP**, where model optimization focuses on extracting critical code segments—such as **call.value**—as indicators of **Re-entrancy vulnerabilities**. However, the authors also acknowledge that many vulnerable **smart contracts do not contain this keyword**, suggesting a potential **bias in feature selection**.

RQ2: *Does Peculiar detect reentrancy vulnerability only based on keywords that are related with reentrancy vulnerability?* We note that most *reentrancy vulnerability* come up with the keywords about external function. However, it should be noted that **not all contracts with these keywords are vulnerable**. Likely, **not all reentrancy vulnerabilities contain these keywords**. Therefore, it is worthy to investigate whether our Peculiar is overfitting to these keywords. We are thus motivated to perform this experiment where we evaluate the performance of Peculiar only on contracts with these keywords.

- **Proposed Modifications to Feature Extraction** - To address this limitation, two alternative strategies were evaluated:
 - Remove the DFG-to-CDFG conversion:
 - This preserves the full **code propagation structure**, allowing the model to **independently infer** vulnerability patterns.
 - Issue:
 - The original DFG is significantly **larger** than CDFG.
 - **Exceeds token length constraints**, leading to **truncation and information loss**.
 - **Conclusion: Ineffective due to excessive context length**.
 - Enhance keyword selection in the preprocessing stage:
 - Instead of relying solely on `call.value`, expand feature selection to other relevant keywords: **"call", "transfer", "send", "balances"**
 - Modify **source code extraction** and **DFG processing** to improve coverage of different vulnerability types.
 - Results:
 - Despite an unintended **filtering mistake**, which retained only **public/external payable functions**, the model achieved **higher accuracy and f1 score**.
 - Notably, while these function types are relatively **few** in the dataset, they still captured key vulnerability patterns.
- **Unexpected Observation** - When functions were **entirely removed**, leaving only **contracts and initialization variables**, model performance remained **high**
 - Implication:
 - While vulnerabilities primarily exist **within functions**, their patterns **extend across entire contracts**.
 - This suggests that **vulnerability characteristics propagate beyond individual functions**, making **contract-wide analysis valuable**.
 - Removing functions **shortened context length**, enhancing model efficiency **without performance degradation**.

- Alternative Preprocessing Strategies - Following these insights, a refined approach was tested:
 - Extract **only functions within contracts**, excluding initialization variables.
 - Apply the following function selection criteria:
 - **Include:** Functions that are **public, external, and payable**
 - **Exclude:** Functions containing:
 - view, constant, pure attributes
 - **Modifiers such as nonReentrant**
 - This targeted filtering **reduced token length**, allowing a **greater volume of meaningful data** to be processed within the Transformer's **context window**.
- Experimental Results - To optimize token efficiency, various **source code and DFG token length combinations** were tested:

Configuration	Code Length (tokens)	DFG Length (tokens)
Initial Setup	384	128
Increased Code Length	448	64
Balanced Approach	372	140
Equal Distribution	256	256

- These variations aimed to find an **optimal trade-off** between preserving **contextual depth** while remaining within the **model's maximum token limit**.

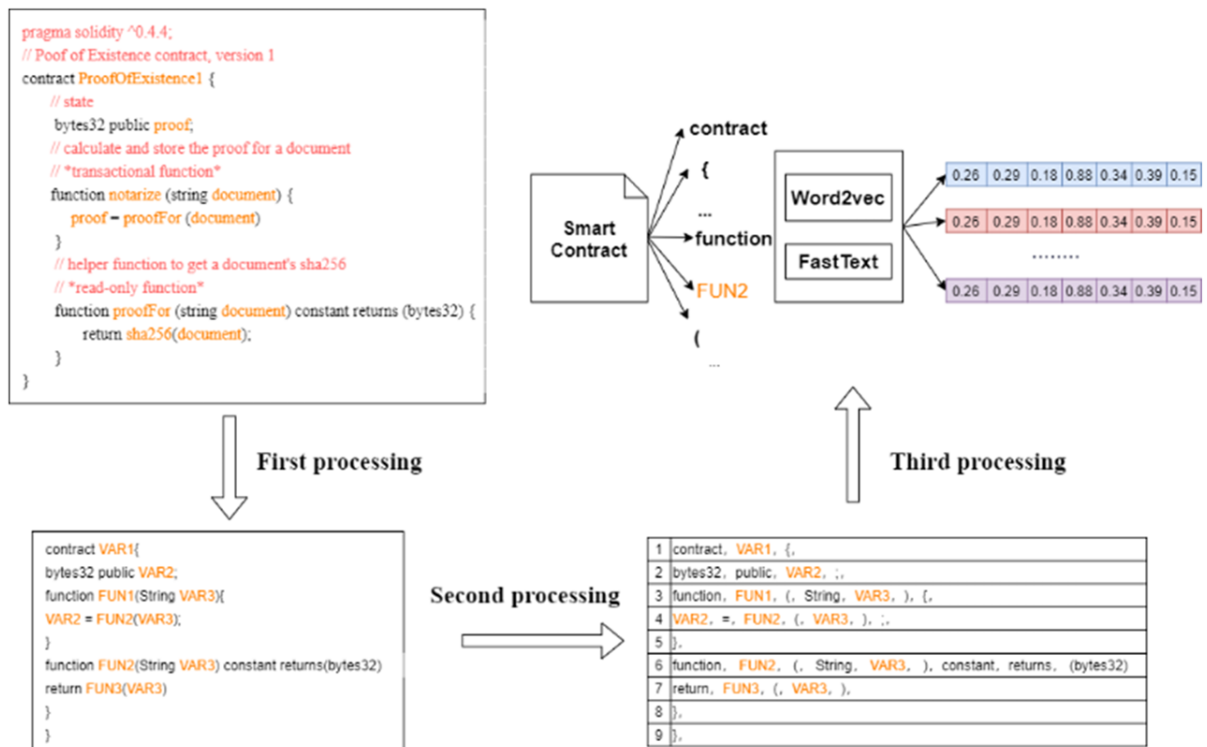
Code Length (tokens)	DFG Length (tokens)	Accuracy	Precision	Recall	F1 Score
384	128	0.8686	0.8662	0.8886	0.8733
448	64	0.8562	0.8887	0.8378	0.8587
372	140	0.8655	0.8921	0.8402	0.8654
256	256	0.8600	0.9017	0.8342	0.8667

○

4.2.2. CBGRU - Sensors 2022 (Q2) – **Status: Stopped due to low quality Journal**

- The CBGRU model integrates the advantages of word embedding techniques (**Word2Vec** and **FastText**) and deep learning architectures (**CNN** and **BiGRU**). The architecture is

structured into four primary stages:



1. Data Processing:

- The **Smart Contract dataset** is preprocessed by filtering out comments and standardizing contract and function names into a unified format, such as VAR and FUN.

2. Word Embedding:

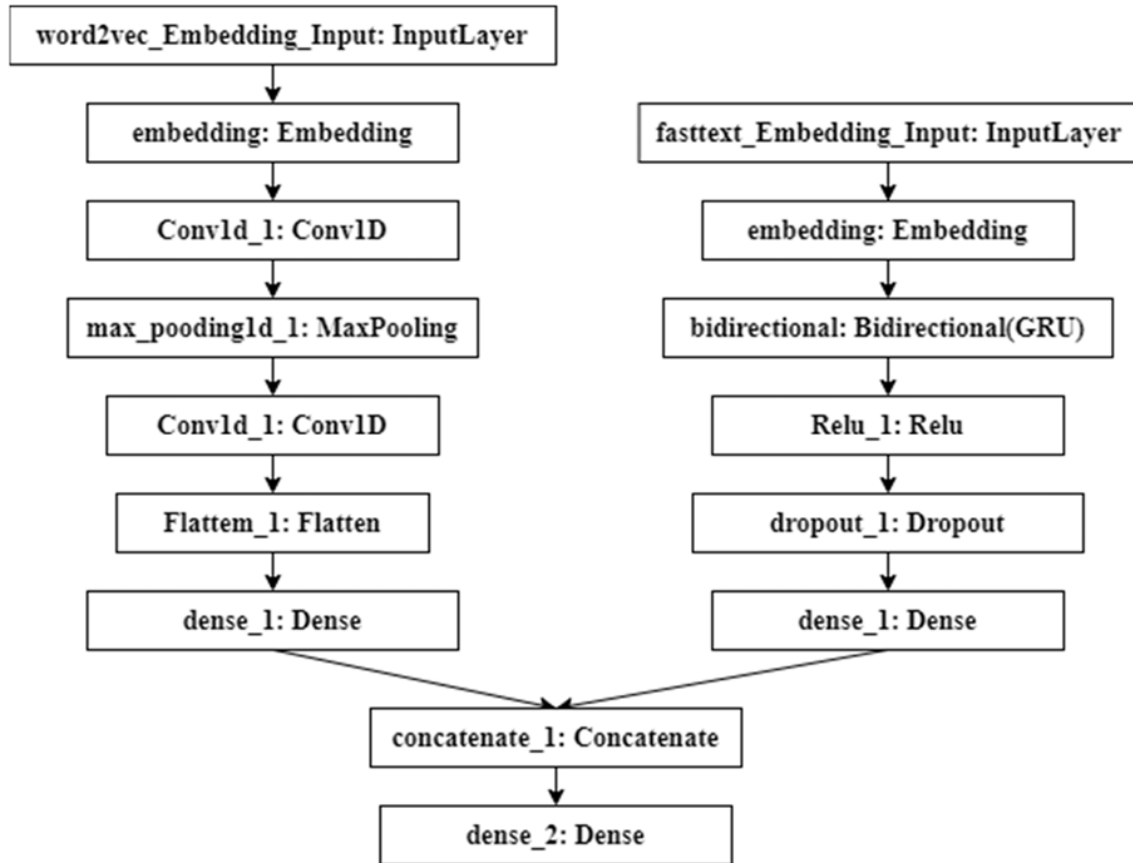
- Smart contracts are transformed into lower-dimensional vector representations using two embedding techniques:
 - **Word2Vec** (CBOW mode)
 - **FastText**

3. Feature Extraction:

- The model employs a dual-branch approach for feature extraction:
 - **CNN** extracts **local** features.
 - **BiGRU** captures **sequential dependencies** in parallel.
 - The extracted features from both branches are then concatenated and passed to the classification layer.

4. Classification:

- The concatenated features are fed through a **fully connected layer**, followed by a **softmax layer** for vulnerability classification.
- This **hybrid model** was chosen after multiple architectural experiments, demonstrating its robustness compared to alternative arrangements.



Model Name	Classification Method One	Embedding Method	Classification Method Two	Embedding Method	Embedding Size	Dropout	Epoch	Accuracy (%)
M1	CNN	Word2Vec	CNN	Word2vec	300	0.5	50	80.78
M2	CNN	Word2Vec	GRU	Word2vec	300	0.5	50	80.33
M3	CNN	Word2Vec	LSTM	Word2vec	300	0.5	50	79.67
M4	CNN	Word2Vec	BiLSTM	Word2vec	300	0.5	50	81.14
M5	CNN	Word2Vec	BiGRU	Word2vec	300	0.5	50	82.10
M6	CNN	Word2Vec	CNN	FastText	300	0.5	50	79.25
M7	CNN	Word2Vec	GRU	FastText	300	0.5	50	80.67
M8	CNN	Word2Vec	LSTM	FastText	300	0.5	50	79.45
M9	CNN	Word2Vec	BiLSTM	FastText	300	0.5	50	83.55
CBGRU	CNN	Word2Vec	BiGRU	FastText	300	0.5	50	85.80
M10-A	CNN	Word2Vec	/	/	300	0.5	50	75.67
M11	BiLSTM	Word2Vec	/	/	300	0.5	50	74.60
M12	BiGRU	Word2Vec	/	/	300	0.5	50	75.56
M13	CNN	FastText	/	/	300	0.5	50	77.65
M14	BiLSTM	FastText	/	/	300	0.5	50	75.04
M15-B	BiGRU	FastText	/	/	300	0.5	50	78.75

- Limitations of CBGRU and Proposal for Improvement: Despite its effectiveness, the **CNN**, **BiLSTM**, and **BiGRU** architectures are relatively outdated and have been largely replaced by **Transformer-based** models. Given this, we propose replacing **BiGRU** with a **Transformer Base Encoder**, specifically **CodeBERT**.
- **Justification for Using CodeBERT:**

- **Pretrained on diverse programming languages, including Java**, which closely aligns with the structure of **Smart Contracts**.
- Offers superior performance compared to generic models like **BERT** and **RoBERTa** for code-related tasks.
- The **Peculiar** preprocessing method is applied to **reduce input sequence length**, optimizing the model's efficiency.
- Performance Evaluation:
 - **Original CBGRU Model Results on the Test Dataset:**

Vulnerability Type	Val Loss	Accuracy	Recall	Precision
Re-entrancy	0.6529	0.7786	0.8011	0.7563
Timestamp Dependency	0.11579	0.7265	0.7628	0.7610
Unhandled Exceptions	0.8068	0.8410	0.9364	0.7523

- **Results After Replacing BiGRU with CodeBERT** (The **F1 Score** metric improved significantly from **0.778** to **0.847**, demonstrating a notable enhancement in performance)

Vulnerability Type	Val Loss	Accuracy	Recall	Precision
Re-entrancy	0.5182	0.8597	0.7792	0.9286

- Why Retain CNN in the Architecture?
 - Hypothesis:
 - CodeBERT, being a **Transformer-based model**, processes input holistically, assigning individual weights to all tokens in a sequence.
 - However, **CNN provides localized feature extraction**, offering a **more fine-grained understanding** of token interactions.
 - Empirical Evidence:
 - When CNN was **removed**, the F1 Score **decreased**, indicating that **CNN embeddings contribute meaningful data to the model**.

Vulnerability Type	Val Loss	Accuracy	Recall	Precision
Re-entrancy	0.3735	0.8163	0.8534	0.7944

- **Token Length and Model Optimization:**
 - **Issue with CBGRU's Token Limitation:**
 - The CBGRU model **extracts only the first 100 tokens** when stacking embeddings.
 - However, an **average smart contract** contains **500–1000 tokens**.
 - This truncation **severely limits the input information**, yet the **original paper does not address this issue**.
 - **Proposed Adjustment:**
 - **Increase token limit** to allow CNN to process a **more comprehensive feature set**.
 - Conduct experiments to isolate the impact of token count on performance.

4.2.3. VulBERTa - IJCNN 2022 (A / A*)

4.2.4. TMP - IJCAI 2020 (A)

4.2.5. AME - IJCAI 2021 (A)

4.2.6.

5. Conclusion

This study demonstrates the effectiveness of using graph-based approaches and transformer models like CodeBERT for vulnerability detection in Solidity smart contracts. By constructing Function Call Graphs and leveraging embeddings from CodeBERT, we achieved significant improvements in classifying common vulnerabilities. Future work could explore the use of larger context models like Longformer or applying hybrid models that combine both global and local contextual features.

6. References

1. SmartBugs Wild Dataset. Available at: <https://github.com/smartbugs/smartbugs-wild>
2. Slither Audited Smart Contracts. Available at: <https://huggingface.co/datasets/mwritescode/slither-audited-smart-contracts>
3. [How to Fine-Tune BERT for Text Classification? \(arxiv.org\)](#)